Manual Técnico - AlertaMujer

Versión: 2.1
Fecha: 28 de octubre de 2026
Audiencia: Desarrolladores, arquitectos de software e ingenieros de DevOps
Nivel técnico: Tecnólogo

Tabla de Contenidos

1. Arquitectura del Sistema
2. Estructura del Código
3. Diagramas de Secuencia
4. Documentación de la API
5. Guía de Desarrollo
6. Base de Datos
7. Seguridad
8. Pruebas e Integración Continua
9. Despliegue
10. Apéndice

---

1. Arquitectura del Sistema

1.1 Visión General

AlertaMujer está compuesto por dos clientes y un backend de microservicios:

- **Backend:** tres microservicios Spring Boot 3.2.0 con Java 17 (**Auth**, **Alert** y **Evidence**), con arquitectura hexagonal en sus nueve módulos. Comparten

una base PostgreSQL 16 versionada con Liquibase.

- **Frontend web:** SPA en Angular 21.2 con TypeScript (portal de la usuaria, panel de administración y página pública de seguimiento).

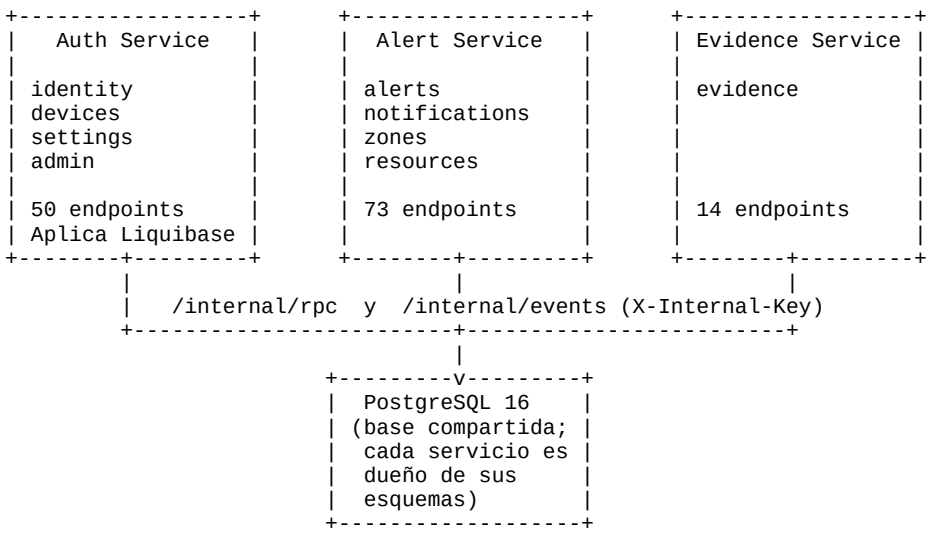
- **Frontend móvil:** app Android con Expo SDK 54, React Native 0.81, React 19.1 y TypeScript 5.9 en modo estricto, compilada con EAS Build.

Servicios externos:

Servicio	Uso
Firebase	Verificación del inicio de sesión con Google y notificaciones push (FCM)
Proveedor de correo por API HTTP (Resend en uso; Brevo alterno)	Aviso de alerta a los contactos, aviso de fin y código de recuperación. Se usa API HTTP porque Render bloquea SMTP
Backblaze B2 (compatible con S3)	Almacenamiento persistente de las evidencias
MapLibre (móvil) y Leaflet con OpenStreetMap (web)	Mapas, sin depender de la facturación de Google Maps
Render	Los tres microservicios (Docker) y la base PostgreSQL
Vercel	Frontend web
EAS Build	APK de Android

El monolito anterior (`backend-monolith/`), el gateway y la carpeta `db/` se eliminaron del repositorio el 25/10/2026.

1.2 Microservicios



Servicio	Módulos	Responsabilidad
Auth	identity, devices, settings, admin	Registro, login (JWT y Google), sesiones, recuperación de contraseña, celulares, configuración del sistema, panel de administración y auditoría
Alert	alerts, notifications, zones, resources	Alertas, ubicación, chat, contactos, avisos por correo y push, seguimiento público, trayectos «Avísame si no llego», protección automática, zonas y recursos de ayuda
Evidence	evidence	Fotos, video y audio en Backblaze B2, huella SHA-256 y retención de 30 días (aviso 5 días antes)

Los tres servicios comparten el módulo **shared-kernel**: seguridad JWT, manejo de errores, contratos `*Api` y cliente RPC.

1.3 Comunicación entre servicios

- **Contratos:** cada módulo publica un contrato (`IdentityApi`, `AlertsApi`, `DevicesApi`, `SettingsApi`, `ZonesApi`, `EvidenceApi`). Si quien lo usa está en el mismo servicio, recibe la implementación local. Si está en otro servicio, recibe un cliente HTTP genérico (`RpcClientFactory`) que llama a `POST /internal/rpc/{Contrato}/{método}`.

- **Errores entre servicios:** viajan con su tipo, así que un 404, 403, 400 o 409 en el servicio remoto llega como el mismo error al que llamó.

- **Eventos:** dentro de un servicio se usan eventos de Spring asíncronos que se publican después de confirmar la transacción. Entre servicios (recuperación de contraseña y celular nuevo) el evento se reenvía por HTTP a `/internal/events`.

- **Seguridad interna:** todo lo que está bajo `/internal/**` exige la cabecera `X-Internal-Key`. Sin ella responde 403.

- **Tiempo de espera del RPC:** 90 segundos, para tolerar el arranque en frío del plan gratuito.

Contrato	Lo expone	Lo consumen
IdentityApi, DevicesApi, SettingsApi	Auth	Alert, Evidence
AlertsApi, ZonesApi	Alert	Auth, Evidence
EvidenceApi	Evidence	Alert (seguimiento público)

1.4 Arquitectura hexagonal

Los nueve módulos siguen el mismo patrón de puertos y adaptadores:

```
com.alertamujer.{modulo}
├── domain
│   └── model              # Modelos puros con las reglas del negocio (sin Spring, JPA ni Lombok)
├── application
│   ├── port
│   │   ├── in             # Casos de uso (interfaces)
│   │   └── out            # Puertos hacia repositorios y otros sistemas
│   └── service            # Implementación de los casos de uso; solo conoce puertos
└── adapter
    ├── in
    │   ├── web            # Controladores REST con DTOs propios
    │   ├── scheduler      # Tareas programadas
    │   ├── eventos        # Escucha de eventos
    │   └── modulo         # Implementación del contrato *Api
    └── out
        ├── persistence    # Entidades JPA (package-private) y conversión al dominio
        └── ...            # Otros servicios, Firebase, correo, almacenamiento, PDF
```

Reglas que se cumplen y se revisan:

- Ningún archivo de `domain/` importa Spring, JPA ni Lombok.
- Ningún caso de uso importa adaptadores, JPA, JWT, Firebase ni OpenPDF.
- Los controladores no devuelven entidades JPA: cada módulo tiene sus DTO de petición y respuesta.
- Cada servicio tiene `HexagonalArchitectureTest` (ArchUnit): falla si el dominio depende de la aplicación, de los adaptadores, de Spring, de JPA o de Lombok, o si un caso de uso depende de un adaptador.

Reglas de negocio que viven en el dominio:

Módulo	Ejemplos de reglas en el dominio
identity	Limpieza de los datos de registro, estados de la cuenta, contraseña de 8 a 100 caracteres, código de recuperación (15 min, 5 intentos, solo se guarda el hash), anonimización
devices	Celular principal para la ubicación diaria, validación de permisos, estados, token push
settings	Duración de la alerta (30 a 1440 min), número de emergencia, límite de contactos, versión mínima
admin	Casos de moderación, eliminación de

Módulo	Ejemplos de reglas en el dominio
	cuenta con dos administradoras, informes (por selección o por filtros), eventos de sesión
alerts	Una alerta activa por usuaria, transiciones de estado, expiración, máximo 5 alertas conservadas, límites del chat, 4 recorridos y 2 con estrella
notifications	Enlace de seguimiento de 256 bits (solo se guarda el hash, vigente hasta 2 h después de terminar), límite de mensajes del contacto
zones / resources	Zonas, reportes de zona, recursos de ayuda y llamadas
evidence	Tipos de archivo aceptados, nombres seguros, revisión

1.5 Arquitectura del Frontend Web

Componentes standalone de Angular con signals y carga diferida, organizados en tres capas:

```
src/app/
├── core/                    # Lo que comparte toda la app (29 servicios)
│   ├── api/                 # servicios.ts: elige el microservicio según la ruta
│   ├── auth/                # AuthService, GoogleAuthService
│   ├── guards/              # authGuard, adminGuard, userGuard
│   ├── interceptors/        # Enrutamiento a microservicios + JWT; cierre ante 401 e inactividad
│   ├── layouts/             # público, usuaria y administración
│   ├── models/
│   ├── services/
│   └── theme/
├── features/
│   ├── landing/             # Página pública
│   ├── auth/                # Login, registro, recuperación
│   ├── dashboard/           # Portal de la usuaria
│   └── admin/               # Panel de administración
└── shared/                  # Componentes, layouts y pipes compartidos
```

- Las rutas de administración existen **solo** bajo `/admin`, protegidas con `authGuard` y `adminGuard`.

- Bootstrap 5.3.8, Tailwind 3.4 (uso menor), Leaflet 1.9.4 y Firebase 13.

- Formularios: uno reactivo (registro) y el resto con plantillas y `ngModel`.

1.6 Arquitectura del Frontend Móvil

React Native con Expo, organizada por funcionalidad con el patrón MVVM mediante hooks `useXxxViewModel`:

```
src/
├── features/               # 23 funcionalidades: view/ + viewModel/ + style/
│   ├── authentication/    # Login, registro, recuperar contraseña
│   ├── activation/        # Cuenta regresiva de la alerta
│   ├── activeAlert/       # Alerta activa y pregunta «¿Sigues en peligro?»
│   ├── contactAlert/      # Seguimiento que ve el contacto y alarma hasta silenciarla
│   ├── shake/             # Alerta en segundo plano al agitar
│   ├── alertChat/         # Chat de la alerta
│   ├── recording/         # Grabación de audio y video por trozos y cola de subidas
│   ├── permissions/       # Permisos y salud de la protección
│   ├── security/          # PIN, PIN de emergencia y detener la protección automática
│   ├── trip/              # «Avísame si no llego»
│   ├── inbox/             # Bandeja de notificaciones (campana)
│   ├── widget/            # Widget de Android
│   ├── version/           # Aviso de versión mínima
│   └── ...
├── navigation/             # AuthNavigator, MainNavigator, Drawer y pestañas
├── services/               # 27 archivos; serviceRoutes.ts elige el microservicio
├── notifications/          # Alarma SOS, silenciado con el volumen, recordatorios
└── components/
modules/
└── shake-sensor/           # Módulo Expo en Kotlin (11 clases)
```

- **Cliente HTTP:** un solo cliente Axios con JWT, tiempo de espera de 20 s, renovación automática del token ante un 401 y enrutamiento a los tres microservicios.

- **Sesión:** tokens en `expo-secure-store`, con respaldo en AsyncStorage si el módulo no está disponible.

- **Nativo:** el módulo `shake-sensor` (Kotlin) tiene el detector de sacudidas (umbral de 4,5 g ajustado al acelerómetro), la grabación de audio (`AudioRecorderService`), la cancelación con el volumen (`VolumeCancelWatcher`), el modo discreto, el trayecto (`TripService`) y la protección automática (`AutoProtection`, `AutoProtectionService`, `GeofenceReceiver` y `BootReceiver`). Sus servicios en primer plano siguen activos con la app cerrada; Notifee muestra las notificaciones.

- **Mapas:** `@maplibre/maplibre-react-native` 11.5.

- **Google:** `@react-native-google-signin` 16.

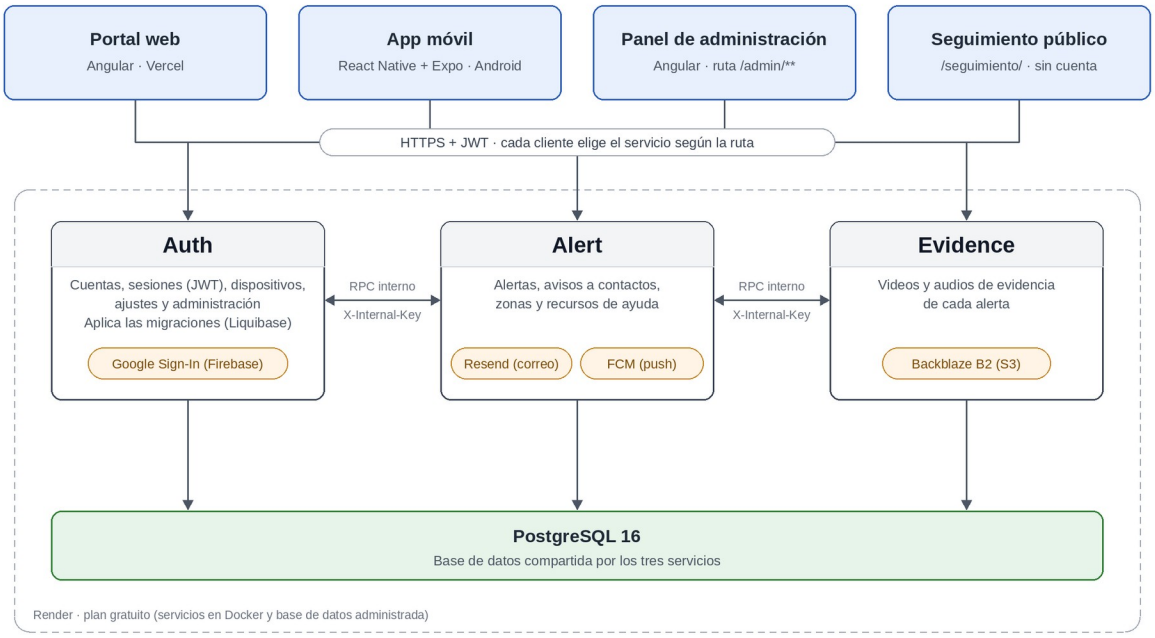
- **Notificaciones:** `@notifee/react-native` 9.1 y `@react-native-firebase/messaging` 26.4.

- **Sensores y cámara:** `expo-sensors` 15 y `expo-camera` 17.

El patrón MVVM se cumple en autenticación, contactos, perfil, ajustes, tutorial, activación, alerta activa, seguimiento del contacto, permisos y grabación. Las vistas del mapa (`MapScreen.tsx` y otras) todavía manejan estado y llamadas a servicios en la vista.

Protección automática: al salir de un lugar frecuente (geocerca de 150 m) `AutoProtectionService` inicia una vigilancia en el servidor (`POST /api/public/auto-protection/start`) y envía una señal con su ubicación cada 2 minutos. Si la usuaria queda quieta 15 minutos fuera de un lugar frecuente, pregunta sin sonido «¿Llegaste bien?»; repite a los 25 minutos y a los 35 llama a `/unconfirmed`, que crea la alerta. En una zona de riesgo pregunta «¿Todo bien?» y, sin respuesta en 5 minutos, llama a `/no-answer`. El servidor crea la alerta si deja de recibir señal 20 minutos. Los botones de la notificación exigen desbloquear el teléfono (Android 12+); con PIN, «Detener» abre la app (`alertamujer://auto-protection/stop`) y el PIN de emergencia llama a `/duress`.

1.7 Flujo completo



---

2. Estructura del Código

2.1 Repositorios

Componente	Repositorio / carpeta	Rama publicada hoy
Frontend móvil	`AlertaMujer/Frontend-movil`	`main` (APK v1.0.1 en GitHub Releases)
Frontend web	`Frond-end-Web/alerta-mujer-web`	`main` (Vercel)
Backend	`Backend-ALERTA-MUJER/microservicios`	`main` (Render)

Cada repositorio tiene solo `develop`, `qa` y `main`. Los cambios entran por pull request a `develop` y se promueven a `qa` y a `main`, que es producción desde el 28/10/2026.

2.2 Estructura del Backend

```
Backend-ALERTA-MUJER/
├── microservicios/
│   ├── pom.xml                      # Maven multi-módulo
│   ├── shared-kernel/               # Seguridad JWT, errores, contratos *Api, cliente RPC
│   ├── auth-service/                # identity, devices, settings, admin + migraciones Liquibase
│   ├── alert-service/               # alerts, notifications, zones, resources
│   └── evidence-service/            # evidence
├── docker-compose.yml               # PostgreSQL, SeaweedFS y los 3 servicios
├── render.yaml                      # 3 servicios + grupo de variables
├── Jenkinsfile                      # Sin servidor verificado; la CI es GitHub Actions
├── .github/workflows/ci.yml         # Pruebas en cada push y pull request
├── ARQUITECTURA_MICROSERVICIOS.md   # Guía vigente de la arquitectura
└── README.md
```

Cada servicio tiene su `Dockerfile` y su `application.properties`. Dentro de cada servicio, los módulos siguen la estructura hexagonal de la sección 1.4.

El `README.md` se actualizó el 25/10/2026. La guía de la arquitectura es `ARQUITECTURA_MICROSERVICIOS.md` y este manual.

2.3 Estructura del Frontend Web

```
Frond-end-Web/alerta-mujer-web/
├── src/
│   ├── app/
│   │   ├── app.config.ts
│   │   ├── app.routes.ts
│   │   ├── core/
│   │   ├── features/
│   │   └── shared/
│   ├── environments/
│   │   ├── environment.ts           # localhost
│   │   └── environment.prod.ts      # apiUrl, alertApiUrl, evidenceApiUrl, firebase
│   └── index.html                   # lang="es"
├── package.json
├── angular.json
├── vercel.json
└── Jenkinsfile
```

2.4 Estructura del Frontend Móvil

```
AlertaMujer/Frontend-movil/
├── App.tsx
├── src/
│   ├── features/
│   ├── navigation/
│   ├── services/
│   ├── contexts/
│   ├── locales/                 # es, en, pt, fr
│   └── components/
├── modules/shake-sensor/        # Módulo Expo en Kotlin (11 clases)
├── scripts/update-env.js        # Escribe en .env las URL con la IP local
├── app.json
├── app.config.js
├── eas.json                     # Perfiles con las URL de los 3 microservicios
├── google-services.json         # FCM e inicio de sesión con Google (no se sube con datos
reales)
├── package.json
└── Jenkinsfile
```

2.5 Patrones de Diseño Implementados

Backend:

- **Arquitectura hexagonal (puertos y adaptadores):** el dominio no depende de la tecnología y cada integración es un adaptador intercambiable.
- **Microservicios con contratos:** interfaces `*Api` con implementación local o cliente RPC remoto.
- **Eventos de dominio:** eventos de Spring después del commit; reenvío HTTP entre servicios.
- **DTO:** objetos propios de petición y respuesta por módulo.
- **Inyección de dependencias** por constructor con Spring.
- **Cadena de filtros:** límite de intentos, clave interna y JWT antes de los controladores.

Frontend Web:

- **Componentes standalone** con carga diferida.
- **Guards** por rol e **interceptores** HTTP.

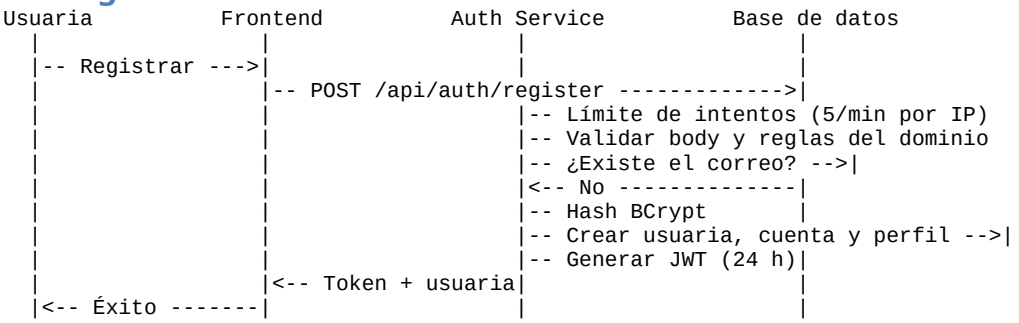
Frontend Móvil:

- **MVVM** con hooks `use*ViewModel`.
- **Context** para el estado global.
- **Navigator** con React Navigation (stack, drawer y tabs).
- **Adapter** en los servicios, que convierten la respuesta del backend al modelo de la app.

---

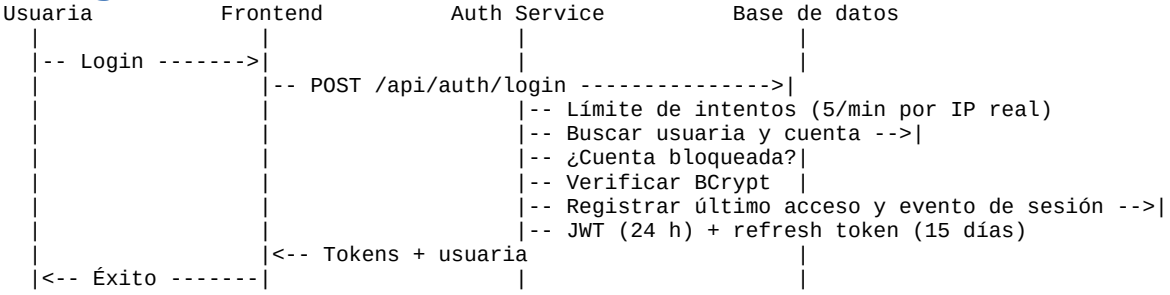
3. Diagramas de Secuencia

3.1 Registro de Usuario



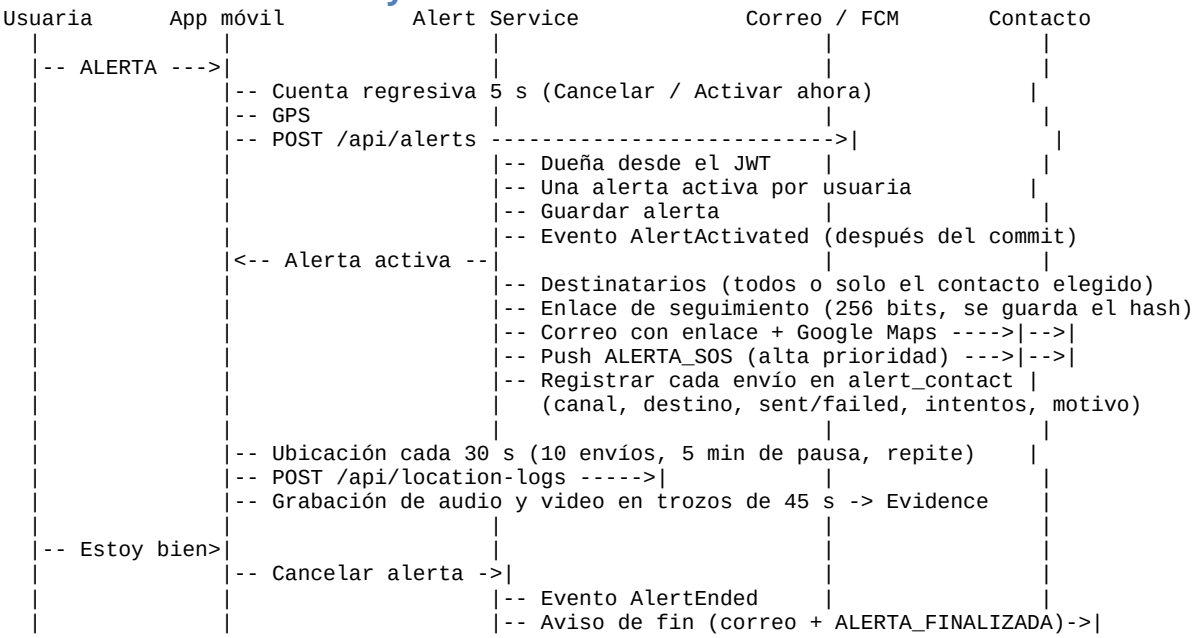
Si el correo ya existe, responde `409 Conflict`.

3.2 Login



Si es un celular nuevo, Auth publica el evento de celular nuevo y los otros celulares de la usuaria reciben un push.

3.3 Activación de Alerta y aviso a contactos



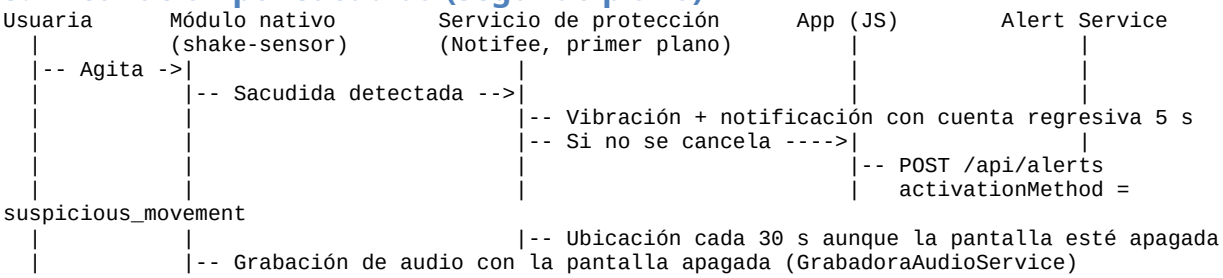
Notas:

- Si no hay conexión, los puntos GPS se guardan en el celular (máximo 100) y se suben en el siguiente envío.

- Si la creación de la alerta falla, la app muestra “No se pudo registrar la alerta en el servidor”. Hoy **no hay reintento automático** (ver sección 10.4).

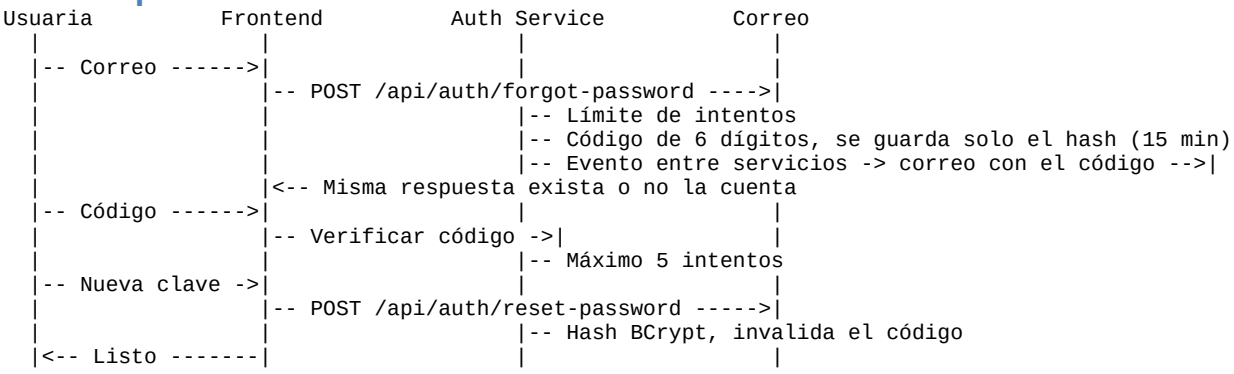
- `ExpiracionAlertasService` expira las alertas que superan la duración configurada y avisa a los contactos.

3.4 Activación por sacudida (segundo plano)

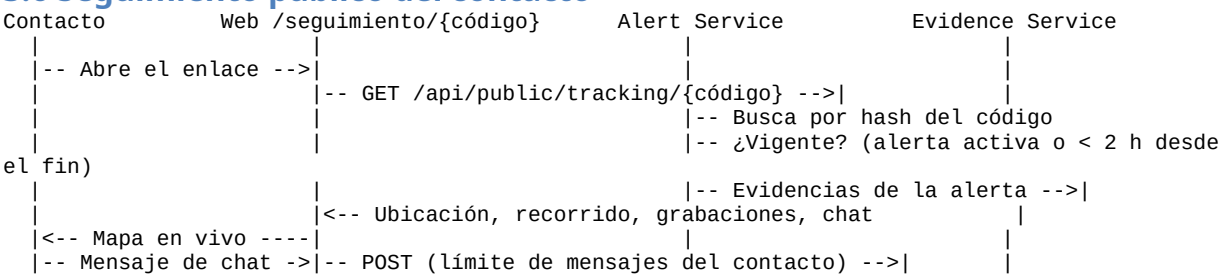


Funciona con la app en segundo plano y la pantalla bloqueada mientras el servicio en primer plano (“Protección activa”) esté corriendo.

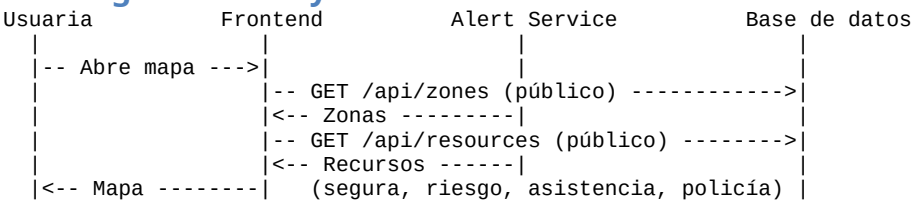
3.5 Recuperación de Contraseña



3.6 Seguimiento público del contacto



3.7 Carga de Zonas y Recursos



---

4. Documentación de la API

4.1 URL base y enrutamiento

No hay API Gateway. Cada cliente elige el servicio según el inicio de la ruta:

Rutas	Servicio
`/api/evidences`	Evidence
`/api/alerts`, `/api/contacts`, `/api/location-logs`, `/api/zones`,	Alert

Rutas	Servicio
`/api/resources`, `/api/saved-routes`, `/api/frequent-locations`, `/api/received-alerts`, `/api/zone-reports`, `/api/resource-calls`, `/api/public/tracking`	
Todo lo demás (`/api/auth`, `/api/users`, `/api/devices`, `/api/admin`, `/api/settings`)	Auth

Servicio	Local (Docker, perfil micro)	Producción
Auth	`http://localhost:8081`	`https://alerta-mujer-auth.onrender.com`
Alert	`http://localhost:8082`	`https://alerta-mujer-alert.onrender.com`
Evidence	`http://localhost:8083`	`https://alerta-mujer-evidence.onrender.com`

La web lee las URL de `environment*.ts` (`apiUrl`, `alertApiUrl`, `evidenceApiUrl`) y el móvil de `EXPO_PUBLIC_API_URL`, `EXPO_PUBLIC_ALERT_API_URL` y `EXPO_PUBLIC_EVIDENCE_API_URL`.

4.2 Autenticación y formato

Los endpoints que no son públicos requieren:

```
Authorization: Bearer {token}
```

Cada servicio valida el JWT con la misma clave, sin llamar a Auth.

Los errores tienen este formato:

```
{
  "status": 404,
  "error": "Not Found",
  "message": "Alert not found with id: 99"
}
```

Códigos usados: 400, 401, 403, 404, 409, 413 y 429. Entre servicios el tipo de error se conserva.

Al migrar a hexagonal se comprobó con pruebas de humo que el JSON de las respuestas no cambió (mismas claves y mismo orden), así que los ejemplos de esta sección valen para la versión desplegada.

4.3 Autenticación (Auth)

POST /api/auth/register

Registro de usuaria (público). Crea la usuaria, su cuenta y su perfil.

```
{
  "nombre": "Maria Garcia",
  "email": "maria.garcia@example.com",
  "password": "********",
  "telefono": "3001234567"
}
```

Validaciones: `nombre` de 2 a 100 caracteres, `email` válido, `password` de 8 a 100 caracteres, `telefono` máximo 20.

Respuesta (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "refreshToken": null,
  "user": {
    "id": 1,
    "firstName": "Maria",
    "lastName": "Garcia",
    "email": "maria.garcia@example.com",
    "telephone": "3001234567",
    "roleId": 1,
    "roleName": "user",
    "documentNumber": null,
    "documentType": null,
    "birthdate": null,
    "createdAt": "2026-10-15T10:30:00"
  }
}
```

Errores: `409` si el correo ya existe; `400` con `fieldErrors` si falla una validación.

POST /api/auth/login

Inicio de sesión (público). Máximo 5 intentos por minuto desde la misma IP.

```
{
  "email": "maria.garcia@example.com",
  "password": "********"
}
```

Respuesta (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "refreshToken": "eyJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": 1,
    "firstName": "Maria",
    "lastName": "Garcia",
    "email": "maria.garcia@example.com",
    "roleId": 1,
    "roleName": "user"
  }
}
```

Errores: `401` con “Invalid credentials” o “Account blocked”; `429` si se superan los intentos.

POST /api/auth/google

Inicio de sesión con Google (público). Acepta el token de Firebase (web) o el ID token de Google (móvil). El token se verifica con firma en el servidor. Si el correo no existe, crea la cuenta. La respuesta es igual a la del login.

```
{ "idToken": "eyJhbGciOiJSUzI1NiIs..." }
```

POST /api/auth/refresh

Entrega un access token nuevo a partir del refresh token (público). El móvil lo usa automáticamente cuando recibe un 401.

```
{ "refreshToken": "eyJhbGciOiJIUzI1NiJ9..." }
```

Respuesta (200): `{ "token": "eyJhbGciOiJIUzI1NiJ9..." }`. Si el refresh token venció, `401`.

Recuperación de contraseña

Tres endpoints públicos bajo `/api/auth`: `forgot-password` (envía el código de 6 dígitos al correo), la verificación del código y `reset-password` (guarda la contraseña nueva). Reglas:

- El código vence a los 15 minutos y admite 5 intentos.
- Solo se guarda el hash del código.
- La respuesta es la misma exista o no la cuenta.
- Tienen el mismo límite de intentos por IP que el login.

4.4 Usuarias y dispositivos (Auth)

- `GET /api/users/me` - Datos de la usuaria autenticada
- `PUT /api/users/me` - Actualizar `firstName`, `lastName`, `telephone`, `documentNumber`, `documentType`, `birthdate` (`yyyy-MM-dd`)
- `GET /api/devices/user` - Dispositivos de la usuaria
- `POST /api/devices` - Registrar dispositivo
- `PUT /api/devices/{id}` - Actualizar (incluye el token push y los permisos)
- `DELETE /api/devices/{id}` - Eliminar
- `/api/settings` - Configuración pública para las apps (por ejemplo, la versión mínima), sin parámetros secretos

Sin token, `GET /api/users/me` responde 403.

4.5 Alertas (Alert)

POST /api/alerts

Crea una alerta (JWT). La dueña se toma del token, no del body. Solo puede haber una alerta activa por usuaria.

```
{
  "deviceId": 1,
  "alertType": "main",
  "activationMethod": "panic_button",
  "status": "active",
  "message": "Alerta SOS activada desde la app",
  "latitude": 2.693470,
  "longitude": -75.329360
}
```

`activationMethod`: `panic_button` (botón), `contact` (desde un contacto, puede avisar solo a ese contacto) o `suspicious_movement` (sacudida).

Respuesta (200):

```
{
  "id": 1,
  "userProfileId": 1,
  "deviceId": 1,
  "status": "active",
  "startedAt": "2026-10-05T10:30:00"
}
```

Al crearse, el servicio avisa a los contactos (sección 3.3).

Otros endpoints de alertas

- `GET /api/alerts/me` - Alertas de la usuaria, de la más reciente a la más antigua
- `GET /api/alerts/{id}` - Una alerta (solo si es de la usuaria)
- `GET /api/alerts/{id}/locations` - Recorrido GPS de la alerta
- `PUT /api/alerts/{id}` - Cambiar el estado (por ejemplo `cancelled`); dispara el aviso de fin
- `GET` y `POST /api/alerts/{id}/messages` - Chat de la alerta
- `GET /api/received-alerts` - Alertas de las personas que tienen a la usuaria como contacto

- `GET /api/public/tracking/{código}` - Seguimiento público (sin JWT, con el código del enlace)

Respuesta típica de una alerta:

```
{
  "id": 1,
  "userProfileId": 1,
  "deviceId": 1,
  "alertType": "main",
  "activationMethod": "panic_button",
  "status": "active",
  "message": "Alerta SOS activada desde la app",
  "latitude": 2.693470,
  "longitude": -75.329360,
  "startedAt": "2026-10-05T10:30:00",
  "endedAt": null,
  "cancelledAt": null
}
```

Si una usuaria intenta ver o modificar la alerta de otra, `403`.

4.6 Contactos de Emergencia (Alert)

- `GET /api/contacts/user/{userProfileId}` - Contactos de una usuaria (JWT)
- `POST /api/contacts` - Crear (el perfil se toma del token)
- `PUT /api/contacts/{id}` - Actualizar (solo la dueña)
- `DELETE /api/contacts/{id}` - Eliminar (solo la dueña)

```
{
  "contactName": "Juan Pérez",
  "telephone": "3111234567",
  "email": "juan@example.com",
  "relationship": "Amigo"
}
```

El teléfono debe tener entre 7 y 20 caracteres, con dígitos y `+` opcional al inicio. El correo es por donde le llega el aviso al contacto. El servidor rechaza un contacto nuevo si la usuaria ya llegó al límite configurado.

`GET /api/contacts` (todos los contactos, solo administración) sigue existiendo; está pendiente retirarlo (ver 7.6).

4.7 Ubicación, lugares y recorridos (Alert)

- `POST /api/location-logs` - Guardar un punto `{latitude, longitude, accuracy, alertId, recordedAt}`
- `GET /api/location-logs/me?limit=50` - Últimos puntos
- `GET /api/location-logs/me/latest` - Último punto
- `GET /api/location-logs/alert/{alertId}` - Puntos de una alerta
- `GET /api/frequent-locations/me`, `POST /api/frequent-locations`, `PUT` y `DELETE /api/frequent-locations/{id}` - Lugares guardados. `riskLevel`: `muy_segura`, `moderada` o `muy_insegura`
- `GET /api/saved-routes/me`, `POST /api/saved-routes`, `PATCH /api/saved-routes/{id}` (`name` o `isStarred`), `DELETE /api/saved-routes/{id}` - Recorridos (máximo 4 por usuaria y 2 con estrella)

En el día a día solo el celular principal reporta la ubicación (cada 50 m o 5 min).

4.8 Zonas y Recursos (Alert)

- `GET /api/zones`, `GET /api/zones/city/{city}`, `GET /api/zones/type/{type}` - Públicos. Tipos: `risk`, `safe`, `help`, `policia`, `asistencia`
- `POST /api/zones`, `PUT /api/zones/{id}`, `DELETE /api/zones/{id}` - Solo administración
- `GET /api/resources`, `GET /api/resources/city/{city}`, `GET /api/resources/type/{type}` - Públicos
- `POST /api/zone-reports`, `PUT /api/zone-reports/{id}/approve`, `PUT /api/zone-reports/{id}/reject` - Reportes de zona (aprobar y rechazar solo administración)
- `POST /api/resource-calls`, `GET /api/resource-calls/me` - Llamadas a recursos

Ejemplo de zona:

```
{
  "id": 1,
  "name": "Parque Santander",
  "zoneType": "risk",
  "riskLevel": "high",
  "description": "Zona de alto riesgo",
  "city": "Neiva",
  "latitude": 2.927300,
  "longitude": -75.289500,
  "radiusMeters": 500.0,
  "isActive": true
}
```

4.9 Evidencias (Evidence)

- `GET /api/evidences/me` - Evidencias de la usuaria
- `POST /api/evidences/upload` - Subir archivo (`multipart/form-data` con `file` y `alertId`, máximo 50 MB)
- `GET /api/evidences/{id}/file` - Ver el archivo (`?download=true` para descargarlo)
- `DELETE /api/evidences/{id}` - Eliminar el registro y el archivo

Cada archivo se guarda en Backblaze B2 con su huella SHA-256. Los archivos de más de 50 MB responden `413`.

4.10 Administración (Auth)

Todos los endpoints de `/api/admin` requieren JWT con rol administrador (`@PreAuthorize("hasRole('ADMIN')")`). Sin ese rol responden `403`.

- `GET /api/admin/users` y `PATCH /api/admin/users/{id}` - Listar, editar y bloquear (con motivo y captura)
- `GET /api/admin/alerts` - Todas las alertas; marcar atendida envía el campo `status`
- `GET /api/admin/user-reports` (y `GET`, `POST`, `PUT`, `DELETE` por id) - Casos de moderación
- `GET /api/admin/audit-logs` - Auditoría con filtros y exportación

- `GET /api/admin/evidences`, `/api/admin/devices`, `/api/admin/zones`, `/api/admin/reports`, `/api/admin/contacts`

Bajo `/api/admin` también están las estadísticas del panel, los informes con exportación PDF y CSV, la eliminación de cuentas con dos administradoras, la configuración del sistema y el estado del servidor. La lista completa está en los controladores `adapter/in/web` de cada módulo.

4.11 Health Check

GET /actuator/health

Público, en cada servicio. Es el que usa Render.

```
{ "status": "UP" }
```

4.12 API interna

- `POST /internal/rpc/{Contrato}/{método}` - Llamadas entre servicios
- `POST /internal/events` - Eventos entre servicios

Exigen la cabecera `X-Internal-Key` con el valor de `INTERNAL_API_KEY`. No son para los clientes.

4.13 Trayectos y protección automática (Alert)

Método y ruta	Acceso	Uso
`POST /api/safe-trips`	Usuaria	Inicia un trayecto con destino y minutos de llegada (5 a 240)
`GET /api/safe-trips/active`	Usuaria	Trayecto activo, si lo hay
`POST /api/safe-trips/{id}/arrived`, `/extend`, `/cancel`	Usuaria	Confirmar la llegada, extender o cancelar
`POST /api/safe-trips/{id}/heartbeat`	Usuaria	Señal de vida con la ubicación (cada 2 minutos)
`POST /api/auto-protection/enable`, `/disable` y `GET /api/auto-`	Usuaria	Encender o apagar la protección automática

Método y ruta	Acceso	Uso
`protection`		en un celular; al encenderla se devuelve una sola vez la llave del celular
`POST /api/public/auto-protection/start`	Celular, cabecera `X-Protection-Key`	Inicia una vigilancia al salir de un lugar frecuente
`POST /api/public/auto-protection/{id}/heartbeat`, `/end`	Celular	Señal de vida y fin al llegar a un lugar frecuente
`POST /api/public/auto-protection/{id}/no-answer`, `/unconfirmed`, `/duress`	Celular	Crea la alerta: sin respuesta a «¿Todo bien?», sin confirmar «¿Llegaste bien?» o con el PIN de emergencia (alerta silenciosa)

Las rutas `/api/public/auto-protection` no usan el JWT: funcionan con la llave propia del celular (en la base solo se guarda su hash en `alert.protection_device`), para que la protección siga aunque la sesión haya vencido. Una cabecera obligatoria ausente responde 400.

5. Guía de Desarrollo

5.1 Requisitos del Entorno

Herramienta	Versión	Para qué
Node.js + npm	20 o superior	Móvil y web
Java JDK / Maven	17 / 3.9	Microservicios sin Docker
Docker + Compose	Reciente	PostgreSQL, SeaweedFS (S3 local) y los 3 servicios en local
Android SDK + JDK 17		APK en local (`eas build --local`)
EAS CLI + cuenta Expo	24.12 o superior	APK en la nube

Herramienta	Versión	Para qué
IDE	IntelliJ IDEA, VS Code o similar	

5.2 Backend en local

```
git clone <url-repositorio-backend> Backend-ALERTA-MUJER
cd Backend-ALERTA-MUJER
cp .env.example .env      # completar las variables (ver Manual de Instalación)
docker compose up --build
```

Levanta PostgreSQL, SeaweedFS como almacenamiento S3 local y los tres servicios en los puertos 8081, 8082 y 8083. Auth aplica las migraciones de Liquibase al arrancar.

Verificar:

```
curl http://localhost:8081/actuator/health
curl http://localhost:8082/actuator/health
curl http://localhost:8083/actuator/health
```

Pruebas:

```
cd microservicios
mvn clean test
```

Los servicios no arrancan si `JWT_SECRET` está vacío o tiene el valor que estuvo publicado en el repositorio. Sin proveedor de correo configurado, los correos solo se escriben en el log, lo que sirve para desarrollo.

5.3 Frontend Web

```
cd Frond-end-Web/alerta-mujer-web
npm install
npm start          # http://localhost:4200
```

Las URL se definen en `src/environments/environment.ts`:

```ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8081',          // Auth
  alertApiUrl: 'http://localhost:8082',     // Alert
  evidenceApiUrl: 'http://localhost:8083',  // Evidence
  firebase: { apiKey: '', authDomain: '', projectId: '', appId: '' },
};
```

Las URL llevan solo el dominio; los servicios agregan `/api/...`. Para producción Angular usa `environment.prod.ts`.

5.4 Frontend Móvil

```
cd AlertaMujer/Frontend-movil
npm install
npm start
```

`npm start` ejecuta `scripts/update-env.js`, que detecta la IP del computador y escribe en `.env` las tres URL (`EXPO_PUBLIC_API_URL`, `EXPO_PUBLIC_ALERT_API_URL` y `EXPO_PUBLIC_EVIDENCE_API_URL`). El celular y el computador deben estar en la misma red Wi-Fi. Para el emulador de Android se usa `10.0.2.2` en lugar de la IP.

Metro usa el puerto 8081 por defecto, el mismo de Auth en local. Si los dos corren en el mismo equipo, Expo ofrece otro puerto; acéptalo.

La app usa módulos nativos (sensor de sacudidas, Notifee, Firebase, Google Sign-In), así que no funciona en Expo Go: hace falta un APK de desarrollo (dev client) o uno de prueba.

Para ver los logs de un APK instalado:

```
adb logcat -v time ReactNativeJS:V AndroidRuntime:E *:S
```

5.5 Agregar funcionalidad en el backend (hexagonal)

1. **Dominio:** crea el modelo en `domain/model` con sus reglas. Sin anotaciones de Spring ni JPA.

2. **Puerto de entrada:** en `application/port/in`, la interfaz del caso de uso.

3. **Puerto de salida:** en `application/port/out`, lo que el caso de uso necesita guardar o consultar.

4. **Servicio:** en `application/service`, implementa el caso de uso usando solo los puertos.

5. **Adaptador de salida:** en `adapter/out/persistence`, la entidad JPA (package-private), el repositorio Spring Data y la conversión entre entidad y modelo.

6. **Adaptador de entrada:** en `adapter/in/web`, el controlador REST con sus DTO de petición y respuesta.

7. **Contrato:** si otro servicio lo necesita, agrega el método al contrato `*Api` del módulo e impleméntalo en `adapter/in/modulo`. El otro servicio lo consumirá por RPC sin cambios adicionales.

8. **Base de datos:** crea un changeset de Liquibase en `auth-service` (es el que aplica las migraciones). Hibernate no crea tablas.

Ejemplo de caso de uso:

```
// application/port/in/CrearNotaUseCase.java
public interface CrearNotaUseCase {
    Nota crear(Long usuariaId, String texto);
}

// application/port/out/NotaRepositorioPort.java
public interface NotaRepositorioPort {
    Nota guardar(Nota nota);
}

// application/service/CrearNotaService.java
public class CrearNotaService implements CrearNotaUseCase {
    private final NotaRepositorioPort repositorio;

    public CrearNotaService(NotaRepositorioPort repositorio) {
        this.repositorio = repositorio;
    }

    @Override
    public Nota crear(Long usuariaId, String texto) {
        Nota nota = Nota.nueva(usuariaId, texto); // las reglas viven en el
dominio
        return repositorio.guardar(nota);
    }
}
```

La dueña de un registro siempre se toma del JWT, nunca del body. Si el endpoint es solo para administración, agrega `@PreAuthorize("hasRole('ADMIN')")` en el controlador.

Changeset de ejemplo:

```
--liquibase formatted sql

--changeset alerta-mujer-team:nota-usuaria
CREATE TABLE alert.nota (
    id              BIGSERIAL PRIMARY KEY,
    user_profile_id BIGINT       NOT NULL REFERENCES identity.user_profile
(id),
    texto           VARCHAR(500) NOT NULL,
    created_at      TIMESTAMP    NOT NULL DEFAULT now()
);
--rollback DROP TABLE alert.nota;
```

5.6 Frontend Web: nueva pantalla

1. `ng generate component features/dashboard/nueva-pantalla`

2. `ng generate service core/services/nueva-pantalla`

3. Agrega la ruta en `app.routes.ts` dentro de los `children` de `dashboard` (o de `admin`) para que herede el layout y los guards:

```
{
  path: 'nueva-pantalla',
  loadComponent: () =>
    import('./features/dashboard/nueva-pantalla/nueva-pantalla')
      .then(m => m.NuevaPantallaComponent)
}
```

Si la ruta del API es nueva, revisa que el prefijo esté en la tabla de enrutamiento de `core/api/servicios.ts`.

5.7 Frontend Móvil: nueva pantalla

1. Crea la vista y su ViewModel en `src/features/nuevaPantalla/` (`view/` y `viewModel/useNuevaPantallaViewModel.ts`). La vista solo pinta; el estado y las llamadas van en el ViewModel.

2. Agrega la ruta al tipo `MainStackParamList` en `src/navigation/types.ts` y regístrala en `MainNavigator.tsx`.

3. Los textos van en `src/locales/` (es, en, pt, fr).

4. Si llamas a una ruta nueva del API, revisa la tabla de `services/servicios.ts`.

5.8 Convenciones de Código

- **Java:** camelCase en variables y métodos, PascalCase en clases, UPPER_SNAKE_CASE en constantes, paquetes en minúsculas, comentarios en español. Las excepciones del dominio se traducen a HTTP en el shared-kernel.

- **TypeScript (web):** archivos en kebab-case, servicios en `core/services`, formato con Prettier.

- **TypeScript (móvil):** pantallas con sufijo `Screen`, servicios `.service.ts`, hooks `use...ViewModel`.

5.9 Flujo de Git

- Ramas por funcionalidad: `feature/*`, `fix/*`, `refactor/*`.

- Integración por merge hacia las ramas de producción.

- Commits en formato Conventional Commits: `tipo(alcance): descripción` (`feat`, `fix`, `refactor`, `docs`, `chore`, `test`, `build`).

- `documentacion`: rama con diagramas, casos de uso, modelo de datos y cronograma.

```
git checkout -b feature/nueva-funcionalidad
git add .
git commit -m "feat(alertas): agregar reintento al crear la alerta"
git push origin feature/nueva-funcionalidad
# Abrir pull request
```

Metodología: desarrollo iterativo e incremental por fases del proyecto formativo SENA.

---

6. Base de Datos

6.1 Esquema

Una sola base PostgreSQL 16 para los tres servicios. Cada servicio es dueño de sus esquemas y no lee tablas de otro servicio: si necesita un dato ajeno, lo pide por el contrato `*Api`. Solo Auth aplica las migraciones.

Esquemas:

- **identity:** autenticación, usuarias, perfiles, dispositivos y recuperación de contraseña
- **alert:** alertas, contactos, envíos a contactos, ubicaciones, evidencias, recorridos, chat, trayectos y protección automática
- **resource:** zonas, reportes de zonas, recursos y llamadas
- **admin:** auditoría, moderación, configuración, informes y solicitudes de eliminación de cuenta
- **notification:** enlaces públicos de seguimiento y sus visitas

Tablas principales:

- **identity:** role, users, account, user_profile, admin_profile, recovery_request, device, device_permission, device_session, alert_activation_setting, user_preference
- **alert:** emergency_contact, alert, alert_contact, alert_message, location_log, notification, evidence, frequent_location, alert_reminder, saved_route, safe_trip, protection_device
- **resource:** zone, zone_report, emergency_resource, resource_call
- **admin:** audit_log, user_report, moderation_action, system_configuration, alert_report, account_deletion_request
- **notification:** tracking_link, tracking_view

En total son 35 tablas y 42 relaciones (migraciones v1.0.0 a v1.6.0, changeset 25). El script, el diccionario y los diagramas están en el Documento 06.

Nota: `users.id` es el que va en el JWT; las alertas, contactos y ubicaciones guardan `user_profile.id`.

6.2 Tablas Principales

identity.users

```
CREATE TABLE identity.users (
    id BIGSERIAL PRIMARY KEY,
    role_id BIGINT NOT NULL DEFAULT 1 REFERENCES identity.role(id),  -- 1
user, 2 administrator
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    telephone VARCHAR(20) UNIQUE,
    email VARCHAR(150) NOT NULL UNIQUE,
    document_number VARCHAR(20) UNIQUE,
    document_type VARCHAR(20),
    birthdate DATE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

identity.account

```
CREATE TABLE identity.account (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL UNIQUE REFERENCES identity.users(id),
    password_hash VARCHAR(255) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'active',  -- active, inactive,
blocked, pending
    last_access TIMESTAMP
);
```

identity.device

```sql
CREATE TABLE identity.device (
    id BIGSERIAL PRIMARY KEY,
    account_id BIGINT NOT NULL REFERENCES identity.account(id),
    device_uuid VARCHAR(255) NOT NULL UNIQUE,
    fcm_token VARCHAR(255),
    brand VARCHAR(50),
    model VARCHAR(100),
    os_name VARCHAR(30) NOT NULL,
    os_version VARCHAR(30),
    app_version VARCHAR(30),
    gps_status VARCHAR(20) NOT NULL DEFAULT 'unknown',  -- active, inactive,
unknown, denied
    status VARCHAR(20) NOT NULL DEFAULT 'active',       -- active, inactive,
blocked, lost
    last_access TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

alert.alert

```sql
CREATE TABLE alert.alert (
    id BIGSERIAL PRIMARY KEY,
    user_profile_id BIGINT NOT NULL REFERENCES identity.user_profile(id),
    device_id BIGINT NOT NULL REFERENCES identity.device(id),
    alert_type VARCHAR(20) NOT NULL DEFAULT 'main',
    activation_method VARCHAR(30) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'active',
    message VARCHAR(500),
    latitude DECIMAL(9,6),
    longitude DECIMAL(9,6),
    started_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ended_at TIMESTAMP,
    cancelled_at TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

alert.alert_contact

Registra cada envío del aviso: contacto, canal (correo o push), destino, estado (`sent`, `failed` o `expired`), número de intentos y motivo del fallo. La usuaria ve el estado por contacto.

alert.emergency_contact

```sql
CREATE TABLE alert.emergency_contact (
    id BIGSERIAL PRIMARY KEY,
    user_profile_id BIGINT NOT NULL REFERENCES identity.user_profile(id),
    contact_name VARCHAR(100) NOT NULL,
    telephone VARCHAR(20) NOT NULL,
    email VARCHAR(150),
    relationship VARCHAR(30),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP
);
```

alert.location_log

```sql
CREATE TABLE alert.location_log (
    id BIGSERIAL PRIMARY KEY,
    user_profile_id BIGINT NOT NULL REFERENCES identity.user_profile(id),
    alert_id INTEGER REFERENCES alert.alert(id),   -- NULL en la ubicación
del día a día
    latitude DECIMAL(9,6) NOT NULL,
    longitude DECIMAL(9,6) NOT NULL,
    accuracy DECIMAL(6,2),
    recorded_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

alert.evidence

```sql
CREATE TABLE alert.evidence (
    id BIGSERIAL PRIMARY KEY,
    alert_id BIGINT NOT NULL REFERENCES alert.alert(id),
    media_type VARCHAR(20) NOT NULL,   -- photo, video, audio, document
    file_url VARCHAR(255) NOT NULL,
    file_name VARCHAR(255),
    content_type VARCHAR(100),
    size_bytes BIGINT,
    sha256 CHAR(64),
    storage_key VARCHAR(100) UNIQUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

alert.saved_route: recorridos con `points` en JSONB (2 a 5000 puntos), `is_starred` e `is_manual`. Un trigger limita a 4 recorridos y 2 con estrella por usuaria.

resource.zone

```
CREATE TABLE resource.zone (
    id BIGSERIAL PRIMARY KEY,
    created_by_admin_id INTEGER,
    name VARCHAR(150) NOT NULL,
    zone_type VARCHAR(20) NOT NULL,                    -- risk, safe, help,
policia, asistencia
    risk_level VARCHAR(20) NOT NULL DEFAULT 'medium',  -- low, medium, high
    description VARCHAR(500),
    address VARCHAR(255),
    city VARCHAR(100) NOT NULL,
    latitude DECIMAL(9,6) NOT NULL,
    longitude DECIMAL(9,6) NOT NULL,
    radius_meters DECIMAL(8,2) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP
);
```

admin.system_configuration: fila única con la duración máxima de la alerta, el número de emergencia, el correo administrativo, el límite de contactos y la versión mínima de la app.

admin.audit_log

```
CREATE TABLE admin.audit_log (
    id BIGSERIAL PRIMARY KEY,
    account_id BIGINT REFERENCES identity.account(id),
    action VARCHAR(100) NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id INTEGER,
    description VARCHAR(500),
    ip_address VARCHAR(45),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

6.3 Índices

Los principales:

```
CREATE INDEX idx_alert_user_profile_created ON alert.alert(user_profile_id,
created_at DESC);
CREATE INDEX idx_alert_active ON alert.alert(started_at DESC) WHERE status =
'active';
CREATE INDEX idx_location_log_user_profile_recorded ON
alert.location_log(user_profile_id, recorded_at DESC);
CREATE INDEX idx_location_log_alert_id ON alert.location_log(alert_id);
CREATE INDEX idx_emergency_contact_user_profile_id ON
alert.emergency_contact(user_profile_id);
CREATE INDEX idx_evidence_alert_id ON alert.evidence(alert_id);
CREATE INDEX idx_zone_city_active ON resource.zone(city) WHERE is_active =
true;
CREATE INDEX idx_audit_log_created_at ON admin.audit_log(created_at DESC);
CREATE INDEX idx_audit_log_entity ON admin.audit_log(entity_type, entity_id);
```

6.4 Migraciones

La base se versiona con **Liquibase**. El archivo principal es `db/changelog/db.changelog-master.yaml` y lo aplica **Auth Service** al arrancar:

- **(raíz):** crea los esquemas `identity`, `alert`, `resource` y `admin`
- **v1.0.0:** tablas de los cuatro esquemas y datos de prueba (solo contextos `dev` y `test`)
- **v1.1.0:** secuencias, índices, restricciones CHECK, `updated_at` automático, auditoría y usuario de la aplicación con permisos limitados
- **v1.2.0:** tabla `saved_route` con su límite de recorridos
- **v1.3.0:** columnas de archivo en `evidence` (nombre, tipo, tamaño, SHA-256)
- **v1.4.0:** roles y configuración base para producción
- **Changesets posteriores:** aviso a contactos, seguimiento, chat, eliminación de cuentas, sesiones y retención

Para un cambio nuevo se crea un archivo en `changesets/` y se incluye al final del master. Nunca se edita un changeset ya aplicado.

Triggers:

- `trg_set_updated_at`: actualiza `updated_at` en cada UPDATE

- `admin.log_row_change`: guarda en `audit_log` los cambios de las tablas principales (solo los nombres de las columnas, no los valores)
- `trg_audit_log_no_update_delete`: impide modificar o borrar `audit_log`
- `enforce_saved_route_limits`: máximo 4 recorridos y 2 con estrella por usuaria

6.5 Tareas programadas

Tarea	Regla
Expiración de alertas	Al cumplirse la duración configurada (30 min por defecto) sin respuesta a «¿Sigues en peligro?», avisa a los contactos y la alerta sigue activa; la marca como `expired` al llegar al tope de 120 min
Reenvío y señal de la alerta	Reenvía el aviso una vez si nadie lo abrió en 5 min y avisa si pasan 5 min sin ubicación
Trayectos y protección automática	Recordatorio 5 min antes de la llegada; alerta si no confirma la llegada o si se pierde la señal (20 min en la protección automática)
Retención de evidencias	Avisa 5 días antes y borra las evidencias de más de 30 días, salvo las de las alertas conservadas (hasta 5 por usuaria)
Retención de ubicaciones	Borra la ubicación del día a día a los 7 días y la de las alertas a los 30

Estas tareas corren dentro de los servicios: solo se ejecutan mientras el servicio esté despierto.

---

7. Seguridad

7.1 Autenticación

- **Contraseñas:** BCrypt, desde un adaptador de salida del módulo identity.
- **JWT:** access token de 24 horas con el id y el rol, y refresh token de 15 días. Firmados con `JWT_SECRET`. `JwtKeyProvider` rechaza un secreto vacío y el valor que estuvo publicado, así que el servicio no arranca con ellos.
- **Validación:** cada servicio valida el JWT con la misma clave, sin llamar a Auth.
- **Google:** el token se verifica con firma en el servidor (`GoogleVerificadorAdapter`).
- **Límite de intentos:** `RateLimitFilter` permite 5 intentos por minuto en login, Google, registro y recuperación, contando la IP real del cliente (`forward-headers-strategy`). Después responde 429.
- **Recuperación:** código de 6 dígitos, se guarda solo el hash, vence a los 15 minutos, 5 intentos y la misma respuesta exista o no la cuenta.

Payload del token:

```
{
  "sub": "1",
  "role": "user",
  "iat": 1696500000,
  "exp": 1696586400
}
```

7.2 Autorización

- **user (role_id 1):** usuaria. En el token: `role: user`.
- **administrator (role_id 2):** administración. En el token: `role: admin`.
- Endpoints de administración con `@PreAuthorize("hasRole('ADMIN')")`.

- Cada usuaria solo ve sus datos: la dueña se toma del JWT en los casos de uso de alertas, contactos, ubicaciones, recorridos y reportes de zona.

- En la web, `authGuard`, `userGuard` y `adminGuard` protegen las rutas; las de administración solo existen bajo `/admin`.

7.3 CORS y API interna

- **CORS:** restringido al dominio de la web en Vercel con `CORS_ORIGIN`. Un origen distinto recibe 403 (comprobado en producción).

- **API interna:** `ClaveInternaFilter` exige `X-Internal-Key` en `/internal/**`.

7.4 Seguimiento público

El enlace de seguimiento tiene 256 bits aleatorios. En la base solo se guarda su hash y deja de funcionar 2 horas después de que termina la alerta. El contacto tiene un límite de mensajes en el chat.

7.5 Datos y auditoría

- Retención automática de evidencias (30 días) y ubicaciones (7 y 30 días).

- Auditoría por triggers en la base y eventos de sesión del panel con IP y origen. El registro de auditoría no se puede modificar ni borrar.

- La eliminación de una cuenta necesita dos administradoras y anonimiza los datos personales.

- El usuario de base de datos de la aplicación solo lee y escribe datos: no puede crear ni borrar tablas, ni modificar la auditoría.

- Restricciones CHECK en la base (formato de correo y teléfono, estados, rango de coordenadas). Si una falla, el backend responde 400 sin mostrar el SQL.

7.6 Clientes

- **Móvil:** tokens cifrados en `expo-secure-store`; los logs del cliente HTTP solo salen en desarrollo (`__DEV__`); tráfico HTTP sin cifrar (`usesCleartextTraffic`) solo en desarrollo local, desactivado en los perfiles de EAS.

- **Web:** token en `localStorage`; cierre de sesión ante 401 y por inactividad.

7.7 Pendientes de seguridad

Tema	Estado
Lo que ve la administración	Corregido: solo el recorrido de alertas activas o atendidas y, por contacto, el estado del envío y el parentesco
Rama antigua con contraseñas y llave de Firebase en el historial	Corregido: rama eliminada y llave inexistente; secreto JWT rotado el 28/10/2026. Falta cambiar la contraseña de la cuenta administrativa que figura en el mensaje de un commit
Ubicación del día a día sin interruptor para desactivarla	Pendiente
Eliminación de cuenta por la usuaria y registro de la aceptación de términos	Corregido (Mi perfil > Eliminar cuenta; `users.terms_accepted_at`)
Token de la web en `localStorage`	Pendiente
La base no cifra columnas en reposo	Pendiente
Quedan `console.*` en el móvil fuera de `__DEV__`	Pendiente
`ACCESS_BACKGROUND_LOCATION`	Se pide solo al activar la protección automática, con explicación previa
Texto «Grabación encriptada» en el tutorial	Pendiente: los archivos no se cifran con clave propia
Accesibilidad del móvil: pocas etiquetas para lectores de pantalla	Pendiente

---

8. Pruebas e Integración Continua

8.1 Resultados (27/10/2026)

Suite	Comando	Resultado
Móvil	`npx jest`	163/163
Móvil (tipos)	`npx tsc --noEmit`	0 errores
Web	`npx ng test --watch=false`	39/39

Suite	Comando	Resultado
Web (build)	`npx ng build`	Compila
Backend	`cd microservicios && mvn clean test`	351/351: shared-kernel 16, Auth 114, Alert 196, Evidence 25

En total, 553 pruebas automatizadas que pasan. GitHub Actions las ejecuta en cada push y pull request desde el 28/10/2026.

8.2 Qué cubre cada suite

- **Móvil (Jest 29, jest-expo 54, React Native Testing Library 13):** detección de sacudidas y gestos del fabricante, servicio de protección, reanudar una alerta, recuperación de contraseña, cola de subidas de la grabación, permisos y salud de la protección, alarma SOS y su silenciado, detener la protección automática con PIN, trayecto, chat, cliente HTTP (401 y renovación), dispositivos, geocodificación, cola de ubicaciones, recorridos, enrutamiento a microservicios y almacenamiento de tokens.

- **Web (Vitest 4 y jsdom):** componentes, reproductor continuo de evidencias y algunos servicios.

- **Backend (JUnit 5 y Mockito):** dominio sin Spring ni base de datos, casos de uso con puertos simulados, adaptadores (JWT, correo, exportador PDF/CSV, cliente RPC), trayectos y protección automática, y `HexagonalArchitectureTest` (ArchUnit).

8.3 Pruebas de humo en Docker (20/10/2026)

Al pasar cada módulo a hexagonal se ejecutó el mismo recorrido contra la versión anterior y la nueva, y se compararon códigos HTTP, claves y orden del JSON:

Prueba	Pasos	Cubre	Resultado
Auth	33	Registro, login, perfil, contraseña, renovación, recuperación, RPC, panel	Idéntico

Prueba	Pasos	Cubre	Resultado
Dispositivos y configuración	35	Alta, uso, permisos, token push, RPC, configuración del panel	Idéntico
Panel de administración	54	Bloqueo, moderación, eliminación con dos administradoras, informes PDF/CSV, auditoría, estado	Idéntico
Alert	30	Contactos, alertas, chat, ubicaciones, recorridos, lugares	Idéntico
Notificaciones	15	Alertas recibidas, seguimiento público, chat del contacto, límite de mensajes	Idéntico
Zonas y recursos	34	CRUD, reportes, llamadas	Idéntico

Además, los correos (aviso de alerta, fin y recuperación) se compararon byte a byte antes y después: idénticos.

8.4 Sin pruebas automatizadas

- Pantallas del móvil y el flujo SOS completo en el teléfono (se prueba a mano en un Samsung SM-A125M y un Infinix X678B); faltan las pruebas del APK v1.0.0 con el trayecto y la protección automática.

- Guards e interceptores de la web; controladores con seguridad por rol.

- Rendimiento, batería y precisión del GPS. No hay medición de cobertura.

8.5 Integración continua (GitHub Actions)

Repositorio	Flujo `.github/workflows/ci.yml`
Móvil	`npm ci`, `npx tsc --noEmit` y Jest
Web	`npm ci`, pruebas y `ng build` de producción
Backend	`mvn -B -ntp test` en `microservicios/`

Corre en cada push y pull request y está en verde desde el 28/10/2026. Los repositorios conservan un `Jenkinsfile` (el del backend prueba los tres microservicios y construye sus imágenes), pero no hay un servidor Jenkins verificado en uso. El APK se compila en local con EAS (`--local`).

---

9. Despliegue

9.1 Producción

Componente	Servicio	Configuración
Web	Vercel	`vercel.json`; despliega con cada push a `main`
Auth, Alert, Evidence	Render (plan gratuito, Docker)	Rama `main`, Root Directory `microservicios`; `render.yaml` describe los 3 servicios y un grupo de variables. UptimeRobot consulta `/actuator/health` cada 5 min
Base de datos	Render PostgreSQL 16	Liquibase la migra al arrancar Auth
Evidencias	Backblaze B2	Compatible con S3, persistente
APK	EAS Build (local) y GitHub Releases	`eas.json` con las URL de los 3 servicios; v1.0.1 del 28/10/2026

El paso a paso está en el **Manual de Instalación**.

9.2 Variables de entorno (solo nombres)

Componente	Variables
Móvil	`EXPO_PUBLIC_API_URL`, `EXPO_PUBLIC_ALERT_API_URL`, `EXPO_PUBLIC_EVIDENCE_API_URL`; `google-services.json` (FCM y Google)
Web	`apiUrl`, `alertApiUrl`, `evidenceApiUrl` y `firebase.*` en `environment*.ts`
Microservicios (todos)	`JWT_SECRET`, `INTERNAL_API_KEY`, `CORS_ORIGIN`, `DB_HOST`, `DB_PORT`, `DB_NAME`, credenciales de la base, `SERVICES_AUTH_URL`, `SERVICES_ALERT_URL`, `SERVICES_EVIDENCE_URL`
Auth y Alert	Archivo de cuenta de servicio de Firebase (Secret File)
Alert	`EMAIL_PROVIDER`, `EMAIL_API_KEY`, `EMAIL_FROM`, `notifications.web-base-url` (base del enlace de seguimiento)
Evidence	Almacenamiento S3 (Backblaze B2): endpoint, bucket y claves

Los valores reales nunca se escriben en el repositorio ni en la documentación.

9.3 Limitaciones del plan gratuito

- Cada servicio se duerme a los 15 minutos sin uso. La primera petición después tarda hasta unos 90 segundos; un monitor de UptimeRobot consulta los tres `/actuator/health` cada 5 minutos y los mantiene despiertos.
- La base gratuita de Render tiene vencimiento.

- Las tareas programadas solo corren mientras el servicio está despierto.

9.4 Monitoreo

```
curl https://alerta-mujer-auth.onrender.com/actuator/health
curl https://alerta-mujer-alert.onrender.com/actuator/health
curl https://alerta-mujer-evidence.onrender.com/actuator/health
```

- Logs: pestaña **Logs** de cada servicio en Render.
- App instalada: `adb logcat -v time ReactNativeJS:V AndroidRuntime:E *:S`.
- Panel de administración: **Configuración** > **Estado del sistema** muestra la conexión a la base, las notificaciones push, las alertas activas y la última revisión de alertas vencidas.

---

10. Apéndice

10.1 Códigos de Error

Código HTTP	Mensaje	Descripción
400	Errores de validación en los campos	Algún campo del body no cumple las validaciones
400	Los datos enviados no son válidos	Un valor no cumple una restricción de la base
401	Invalid credentials	Credenciales inválidas
401	Account blocked	Cuenta bloqueada
401	Access Token expirado	Token expirado (el móvil lo renueva solo)
403	No tienes permisos para acceder a este recurso	Rol sin permiso, recurso de otra usuaria o falta la clave interna
404	{Recurso} not found with id: {id}	Recurso no encontrado
409	Email already exists	Correo ya registrado
409	Solo puedes tener 4 recorridos guardados	Límite de recorridos
409	Conflict	Ya hay una alerta activa o se superó el límite de

Código HTTP	Mensaje	Descripción
		contactos
413	El archivo es demasiado grande (máximo 50 MB)	Evidencia muy pesada
429	Demasiados intentos	Más de 5 intentos por minuto en autenticación
500	Ocurrió un error inesperado	Error interno

10.2 Estados y valores

Estados de alerta: `active`, `resolved` (atendida), `cancelled`, `expired`.

Métodos de activación en uso: `panic_button`, `contact`, `suspicious_movement`.

Estados de envío a contactos (`alert_contact`): `sent`, `failed`, `expired`.

Notificaciones push: `ALERTA_SOS` (abre el seguimiento con el código) y `ALERTA_FINALIZADA`.

Estados de cuenta: `active`, `inactive`, `blocked`, `pending`.

Estados de dispositivo: `active`, `inactive`, `blocked`, `lost`.

Estados de reporte de zona: `pending`, `approved`, `rejected`.

Niveles de ubicación frecuente: `muy_segura`, `moderada`, `muy_insegura`.

10.3 Trazabilidad con el SRS

Según la revisión del código del 20/10/2026, de los 48 subrequisitos funcionales del SRS hay 37 implementados, 8 parciales y 3 no implementados. Esa cifra es histórica. Según el Documento 02 versión 2.6 (28/10/2026), de los 103 requisitos funcionales 99 están implementados, 3 en desarrollo y 1 retirado (la activación desde la pantalla de bloqueo). El widget está implementado.

10.4 Pendientes técnicos

#	Pendiente	Motivo
1	Cola local para crear la alerta sin red	Hoy la app muestra el estado y permite reintentar, pero sin red

#	Pendiente	Motivo
		la alerta no se registra
2	Pasar las vistas del mapa a ViewModels	`MapScreen.tsx` mezcla vista y lógica
7	Unificar los servicios duplicados de zonas y evidencias en la web	Mantenimiento
8	Quitar el modo demo del móvil (pantalla Index y botón temporal del seguimiento)	Código muerto
9	Corregir el texto «Grabación encriptada» del tutorial o cifrar los archivos	Los archivos no se cifran con clave propia
10	Tiempo de espera global en el interceptor de la web	Peticiones colgadas con el servidor dormido
12	Probar un build `production` (AAB)	Nunca se ha ejecutado
13	Guardar las pruebas de humo en el repositorio (`scripts/smoke/`)	Reutilizarlas

10.5 Recursos Adicionales

- Documentación de Spring Boot
- Documentación de Liquibase
- Documentación de Angular
- Documentación de React Native
- Documentación de Expo

- Documentación de PostgreSQL
- Documentación de Render
- Documentación de Backblaze B2

Fin del Manual Técnico

Este manual está dirigido a quienes desarrollan y mantienen AlertaMujer. Para el uso de la aplicación, consulte el Manual de Usuario.